

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- Analyze the CSS properties responsible for layout overlapping issues.
- Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I P Sai bhargav(192411049) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Responsive Layout Design

(a) Analyze the possible reasons for lack of responsiveness in the existing layout.

Possible Reasons:

- Fixed width (width: 1000px) instead of percentage or flexible units.
- Absence of the viewport meta tag.
- No media queries for different screen sizes.
- Using floats instead of Flexbox or CSS Grid.
- Images without responsive sizing.
- Improper spacing and alignment.

(b) Design a responsive webpage layout using CSS Flexbox

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Layout</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<header>Retail Store</header>

<nav>
  <a href="#">Home</a>
  <a href="#">Products</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
</nav>

<div class="container">
  <aside>Sidebar</aside>
  <main>Main Content Area</main>
</div>

<footer>© 2026 Retail Store</footer>

</body>
</html>
```

CSS:

```
*{  
  margin:0;  
  padding:0;  
  box-sizing:border-box;  
}
```

```
body{  
  font-family:Arial;  
}
```

```
header,nav,footer{  
  background:#333;  
  color:white;  
  padding:20px;  
  text-align:center;  
}
```

```
nav{  
  display:flex;  
  justify-content:center;  
  gap:20px;  
}
```

```
nav a{  
  color:white;  
  text-decoration:none;  
}
```

```
.container{  
  display:flex;  
  gap:20px;  
  padding:20px;  
}
```

```
aside{  
  flex:1;  
  background:#ddd;  
  padding:20px;  
}
```

```
main{  
  flex:3;  
  background:#eee;  
  padding:20px;  
}
```

(c) Demonstrate layout adaptation using media queries

/* Tablet */

```
@media(max-width:768px){
```

```
.container{  
    flex-direction:column;  
}
```

```
nav{  
    flex-wrap:wrap;  
}
```

```
}
```

/* Mobile */

```
@media(max-width:480px){
```

```
nav{  
    flex-direction:column;  
}
```

```
header,footer{  
    font-size:18px;  
}
```

```
}
```

(d) Implement spacing, alignment and distribution

- Used **Flexbox** for flexible alignment.
- Added **gap** between navigation items.
- Used **padding** for proper spacing.
- Applied **box-sizing: border-box**.
- Used **justify-content** and **flex** properties for equal distribution.

(e) Evaluate the responsive design

Advantages

- Works on desktop, tablet, and mobile.
- Proper spacing and alignment.
- Easier maintenance.
- Better user experience.
- Improved readability.

Further Improvements

- Use CSS Grid for complex layouts.
- Optimize images.
- Use relative units (rem, %, vw).
- Add responsive typography.
- Test on multiple browsers and devices.

Experiment 2: Form Validation using JavaScript

(a) Analyze the issues in the existing validation

Problems:

- Invalid email addresses are accepted.
- Phone numbers may contain letters.
- Empty fields are submitted.
- No real-time validation.
- Poor user feedback.

(b) Design input fields

```
<form>
```

```
<label>Email</label>
```

```
<input type="email" id="email" required>
```

```
<span id="emailError"></span>
```

```
<label>Phone Number</label>
```

```
<input type="tel" id="phone" required maxlength="10">
```

```
<span id="phoneError"></span>
```

```
<button type="submit">Register</button>
```

```
</form>
```

(c) Implement JavaScript validation

```
const email=document.getElementById("email");
```

```
const phone=document.getElementById("phone");
```

```
email.addEventListener("input",validateEmail);
```

```
phone.addEventListener("input",validatePhone);
```

```
function validateEmail(){
```

```
let pattern=/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;
```

```
if(pattern.test(email.value))
```

```
document.getElementById("emailError").innerHTML="";
```

```
else
```

```
document.getElementById("emailError").innerHTML="Invalid Email";

}

function validatePhone(){

let pattern=/^[0-9]{10}$/;

if(pattern.test(phone.value))
document.getElementById("phoneError").innerHTML="";
else
document.getElementById("phoneError").innerHTML="Enter 10 digit number";

}
```

(d) Display real-time error messages

The above JavaScript validates every time the user types.

Example:

Email:

abc@gmail.com ✓

Phone:

9876543210 ✓

If invalid:

Invalid Email

Enter 10 digit number

(e) Evaluate the validation system

Advantages

- Immediate feedback.
- Prevents invalid submissions.
- Better user experience.
- Reduces server-side validation load.
- Easy to maintain.

Enhancements

- Validate on the server as well.
- Use HTTPS.
- Protect against SQL Injection and XSS.
- Add password strength validation.
- Implement CAPTCHA for bots.

Experiment 3: CSS Layout Debugging

(a) Analyze CSS properties causing overlap

Common causes:

- Fixed widths.
- position:absolute.
- Negative margins.
- Large fixed heights.
- Overflow hidden.
- Lack of responsive layout.

(b) Explain the CSS Box Model

The CSS Box Model consists of:

- **Content** – Actual content inside the element.
- **Padding** – Space between content and border.
- **Border** – Surrounds the padding.
- **Margin** – Space outside the border.

(c) Debug using positioning techniques

Incorrect CSS

```
.box1{
position:absolute;
left:100px;
}
```

```
.box2{
position:absolute;
left:100px;
}
```

Both boxes overlap.

Correct CSS

```
.container{
display:flex;
gap:20px;
flex-wrap:wrap;
}
```

```
.box{
width:250px;
height:150px;
background:lightblue;
}
```

(d) Modify CSS to prevent overlapping

```
*{
box-sizing:border-box;
```

```
}
```

```
.container{  
display:flex;  
flex-wrap:wrap;  
justify-content:space-around;  
gap:20px;  
padding:20px;  
}
```

```
.box{  
flex:1 1 250px;  
padding:20px;  
background:#ddd;  
}
```

This allows elements to wrap automatically when the screen size decreases.

(e) Evaluate the corrected layout

Advantages

- No overlapping during resizing.
- Flexible and responsive layout.
- Better readability.
- Consistent spacing.
- Easier maintenance.

Best Practices

- Use Flexbox or CSS Grid instead of absolute positioning.
- Avoid fixed widths and heights where possible.
- Use box-sizing: border-box.
- Apply media queries for different screen sizes.
- Test layouts across multiple browsers and devices.